



République Tunisienne



Ministère de l'Enseignement Supérieur et de la Recherche Scientifique  
Université de Gabès  
Faculté des Sciences de Gabès



---

## Département d'Informatique

### Projet académique

*Dans le cadre du module Fondements de l'Intelligence Artificielle (Programmation IA)*

Réalisé par:

**Yosri Khedher & Omrane Khedher**

Élaboré par Dr.Mounira Sassi

---

# Optimisation du Routage des Données dans un Réseau en utilisant l'Algorithme A\*

LGLSI-2  
2025-2026

# Contexte

---

Dans la programmation en intelligence artificielle, les problèmes de recherche de chemin sont fondamentaux et visent à trouver une solution optimale dans un espace d'états.

Le routage réseau peut être modélisé comme un problème de recherche entre une source et une destination, en tenant compte des obstacles et des coûts.

Dans ce projet, le réseau est modélisé sous forme de grille afin d'appliquer l'algorithme de recherche  $A^*$  pour trouver le chemin optimal.

# Motivation

---

résoudre un problème concret de routage.

Plus précisément, nous utilisons l'algorithme  $A^*$ , qui est un algorithme de recherche informée basé sur une fonction d'évaluation combinant :

- le coût réel du chemin parcouru
- une estimation du coût restant (heuristique)

Ce projet permet de :

- Comprendre le fonctionnement de l'algorithme  $A^*$
- Modéliser un problème réel en termes d'états, actions et coûts
- Appliquer une heuristique pour améliorer les performances

# Problématique

---

Comment optimiser le routage des données dans un réseau en déterminant le chemin le plus efficace entre une source et une destination, tout en tenant compte des obstacles, des distances et des variations de débit, à l'aide de l'algorithme  $A^*$  ?

# Objectifs du projet

---

L'objectif de ce projet est de concevoir un modèle de routage permettant de déterminer le chemin optimal entre une source et une destination dans un réseau.

Plus précisément, il s'agit de :

- modéliser le réseau sous forme de grille
- appliquer l'algorithme  $A^*$  pour la recherche de chemin
- minimiser le coût de transmission
- éviter les obstacles présents dans le réseau

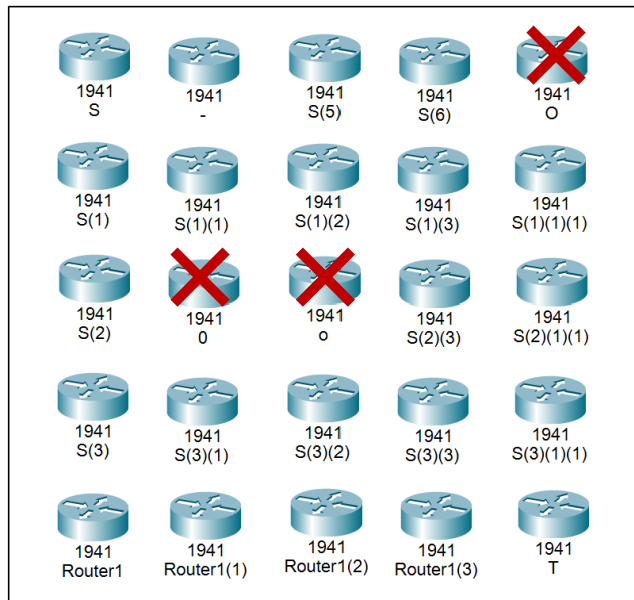
# Modélisation du problème

---

Dans cette partie, on décrit le problème en termes d'IA :

- **États** : position de l'agent dans la grille  $(x, y)$
- **État initial** : position de départ S  $(0,0)$
- **État final** : position cible T  $(n-1,n-1)$
- **Actions** : déplacements (haut, bas, gauche, droite)
- **Contraintes** : éviter les obstacles et rester dans la grille
- **Fonction de coût** : dépend du chemin parcouru (et éventuellement du débit)
- **Heuristique** : distance de Manhattan

| Élément | Signification                             |
|---------|-------------------------------------------|
| S       | routeur source (envoie les données)       |
| T       | routeur destination                       |
| -       | routeurs intermédiaires                   |
| O       | routeurs indisponibles (panne/congestion) |



**Remarque :** Le réseau est modélisé comme un espace d'états discret, où chaque routeur correspond à un nœud dans cet espace.

**Remarque :** Dans un réseau réel, les routeurs sont espacés et reliés par différents types de connexions. La distance et le type de câblage influencent directement le débit et la bande passante. Dans notre projet, ces caractéristiques sont modélisées de manière simplifiée sous forme de grille.

# Implémentation

---

Le projet a été implémenté en Python sous Google Colab.

L'implémentation comprend :

- la création du réseau sous forme de grille
- l'intégration des coûts de transmission
- l'application de l'algorithme A\*
- la simulation du chemin optimal

# I. Algorithme A\* — Recherche dans un graphe:

## Principe général :

L'algorithme A\* est un algorithme de recherche informée utilisé pour trouver le chemin optimal entre une source et une destination.

## Fonction d'évaluation :

$$f(n) = g(n) + h(n)$$

$g(n)$  : coût réel depuis la source

$h(n)$  : estimation heuristique vers la destination

## Étapes de l'algorithme :

### 1-Initialisation

Déclarer deux listes :

- Open : nœuds à explorer
- Closed : nœuds déjà explorés

Ajouter le nœud initial dans Open

### 2-Sélection du nœud

Choisir le nœud ayant la plus petite valeur  $f(n)$

### 3-Vérification du but

Si le nœud courant est la destination :

→ retourner le chemin optimal

### 4-Exploration des successeurs

Pour chaque voisin du nœud courant :

Calculer :

$$g(n') = g(n) + \text{coût}$$

Calculer :

$$f(n') = g(n') + h(n')$$

Ajouter le nœud dans Open si nécessaire

### 5-Répéter

Continuer jusqu'à atteindre la destination ou jusqu'à ce que Open soit vide

## Heuristique utilisée :

Distance de Manhattan :

$$h(n) = |x - x^T| + |y - y^T|$$

## II. Fonction coût:

Dans notre projet, chaque lien entre deux routeurs possède un coût de transmission. Ce coût dépend du débit du lien et la distance entre les routeurs :

$$\text{coût} = \text{distance} / \text{débit}$$

- grande distance → coût augmente
- faible débit → coût augmente
- petite distance + bon débit → coût faible
- grande distance + débit élevé → le débit peut compenser l'augmentation du coût
- petite distance + faible débit → coût peut rester élevé malgré la courte distance

L'algorithme A\* privilégie donc les chemins les plus performants.

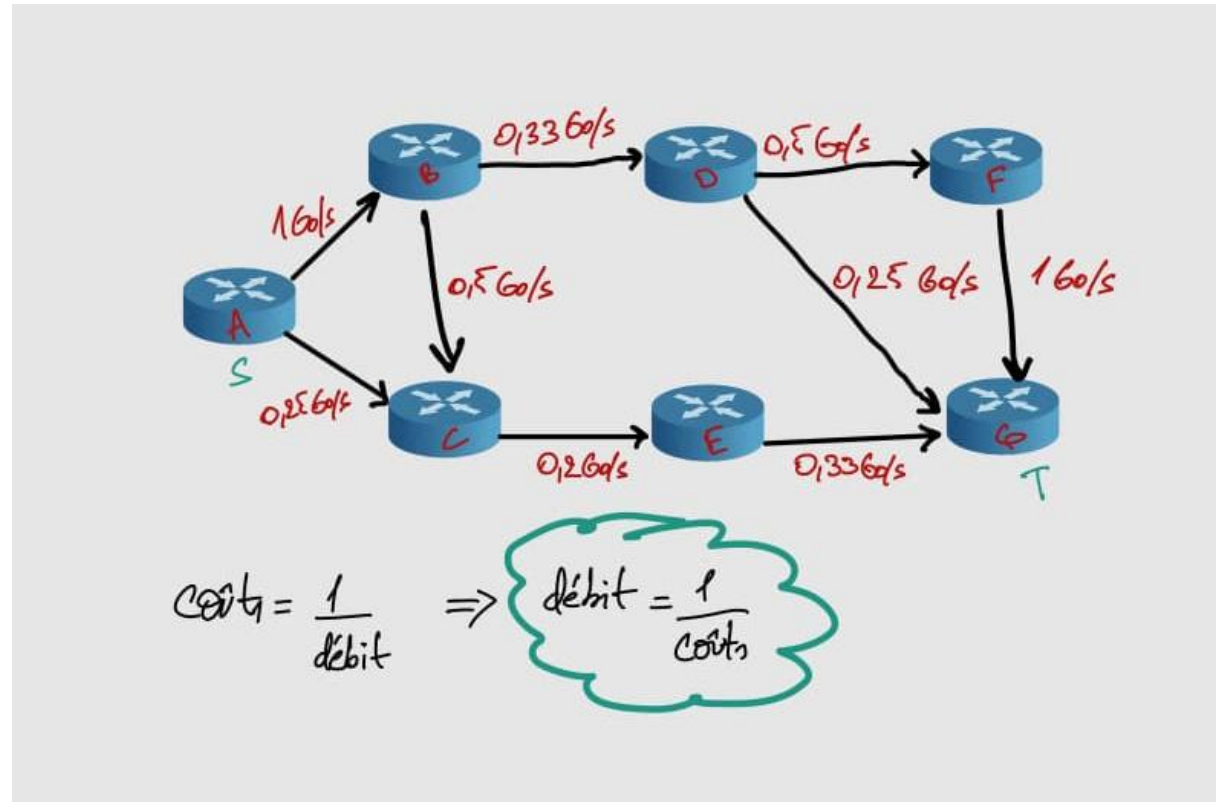
## III. Obstacles réseau :

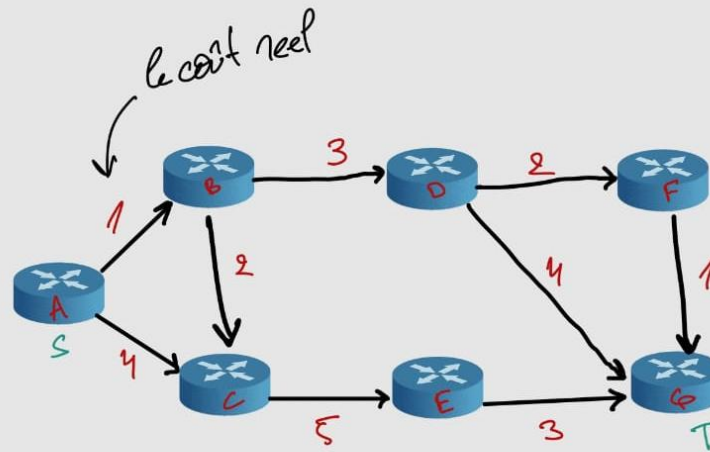
Les obstacles représentent :

- des pannes
- des liens indisponibles
- des routeurs hors service

Ces obstacles ne peuvent pas être traversés par l'algorithme.

#### IV. Application de l'algorithme A\* sur un graphe de routeurs:





Sachant: que  $F(n) = g(n) + h(n)$

et les valeurs heuristiques sont

$$A = 5$$

$$B = 6$$

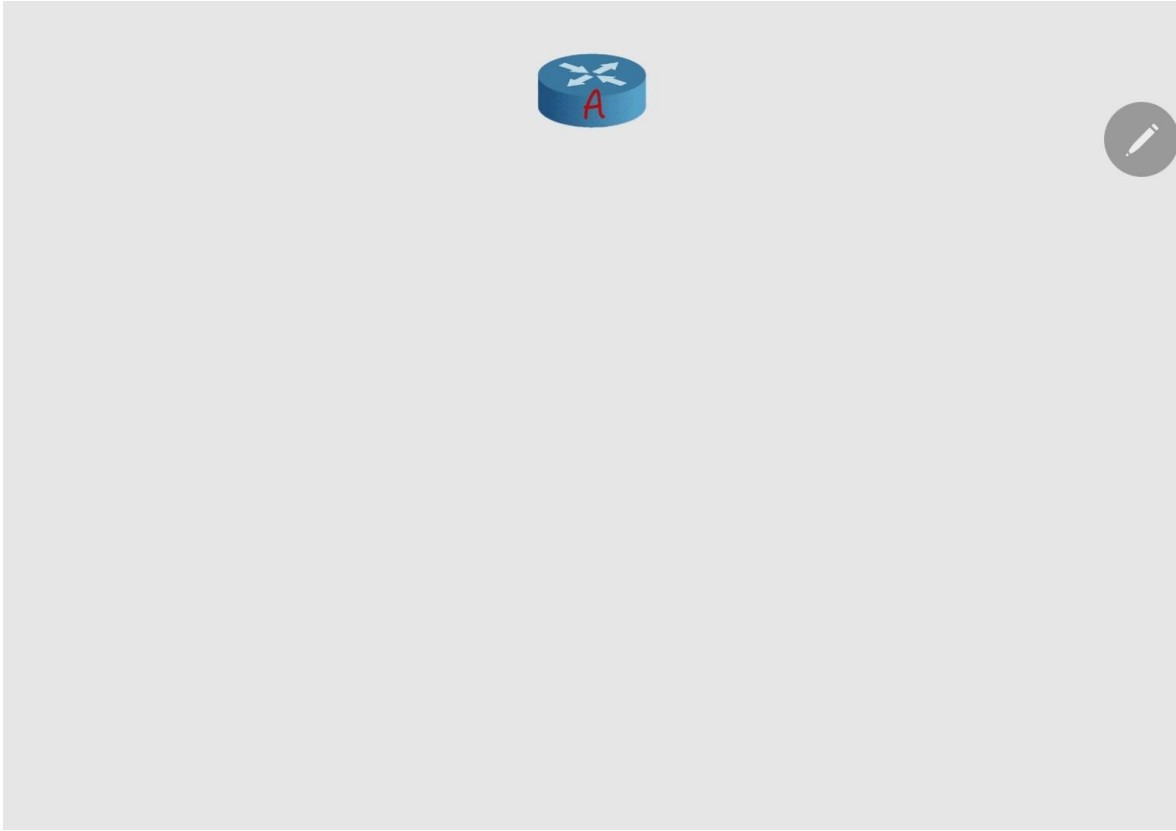
$$C = 4$$

$$D = 3$$

$$E = 3$$

$$F = 1$$

$$G = 0$$

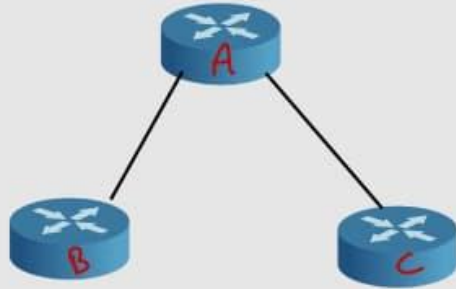


| Etape | Open List | Closed List |
|-------|-----------|-------------|
| 0     | A         | ∅           |
|       |           |             |
|       |           |             |
|       |           |             |
|       |           |             |
|       |           |             |

$$f(B) = g(B) + h(B)$$

$$= 1 + 6$$

$$= 7$$

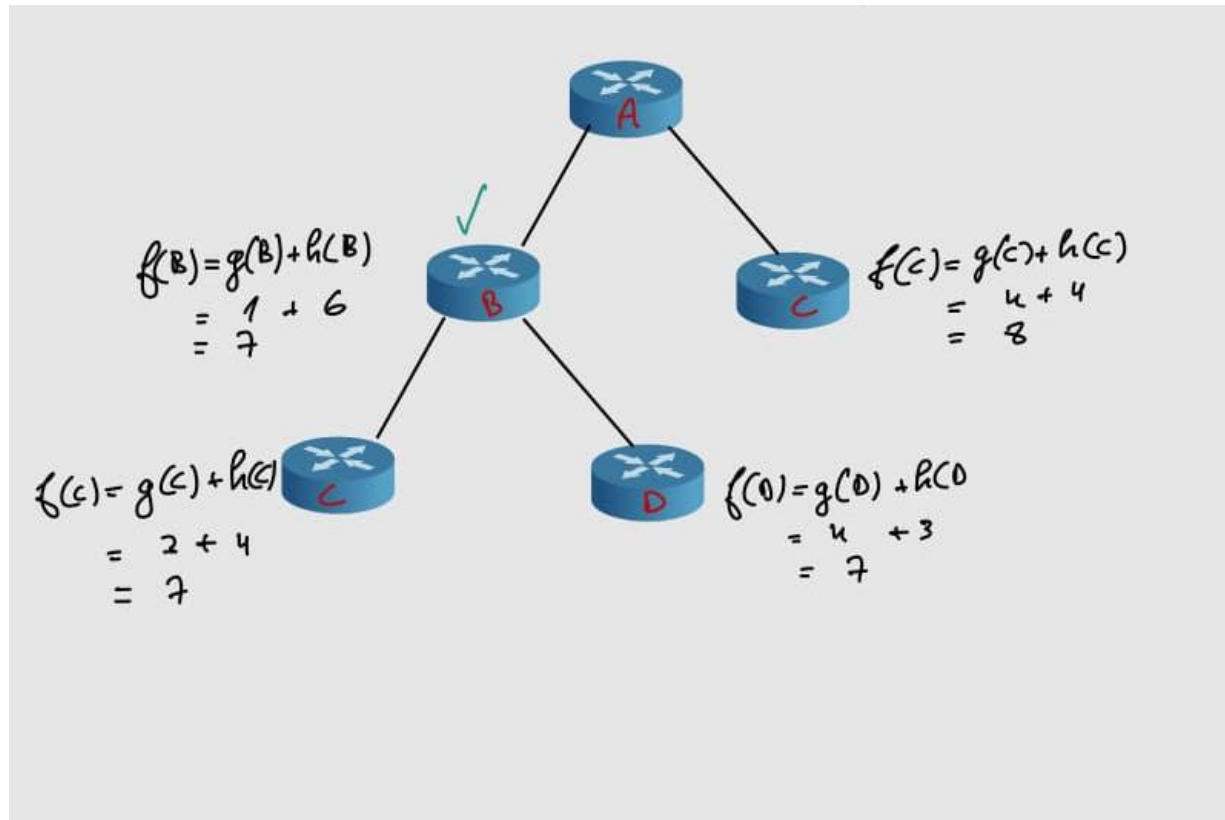


$$f(C) = g(C) + h(C)$$

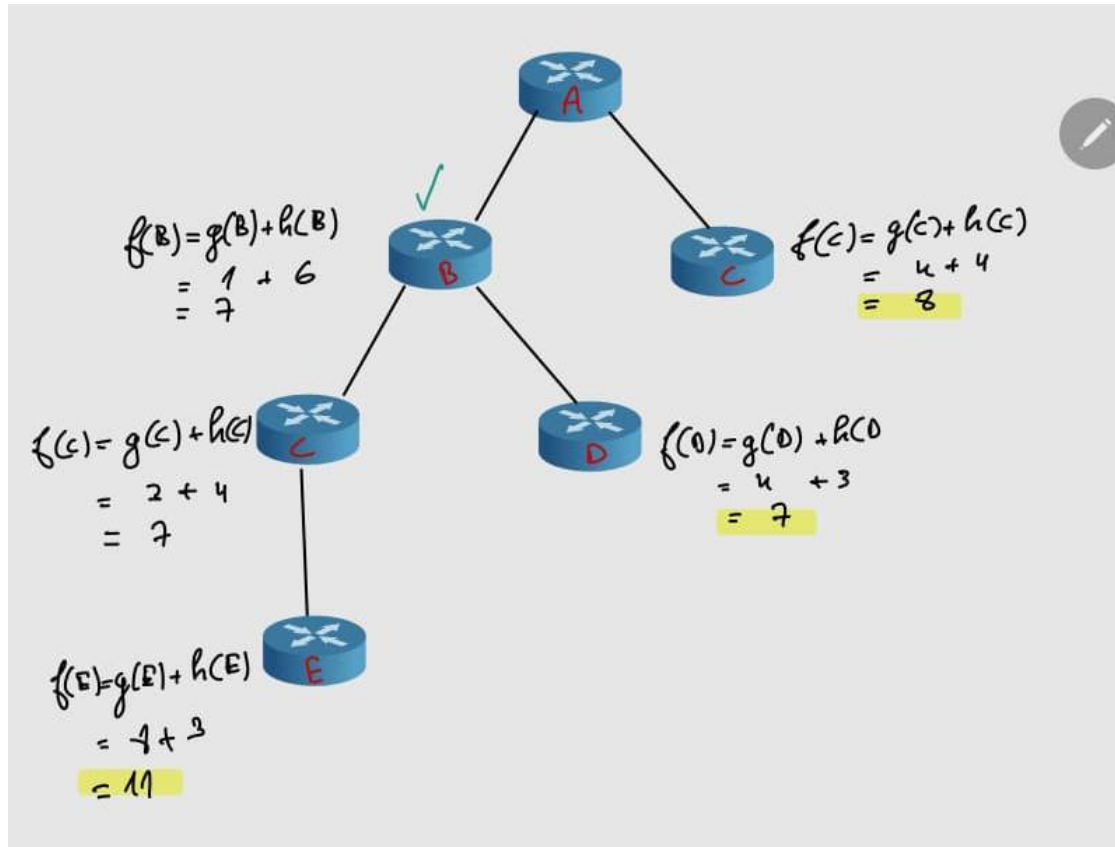
$$= 4 + 4$$

$$= 8$$

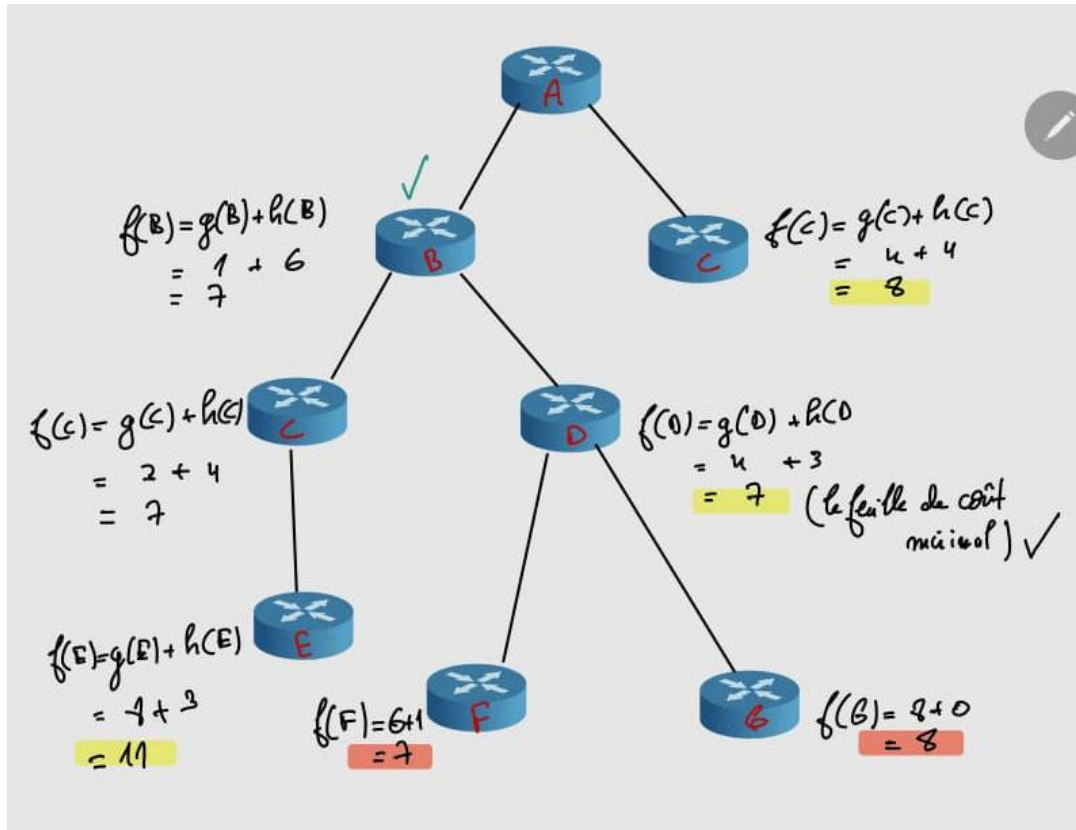
| Etape | Open List  | Closed List |
|-------|------------|-------------|
| 0     | A(0)       | ∅           |
| 1     | B(7), C(8) | A           |
|       |            |             |
|       |            |             |
|       |            |             |
|       |            |             |



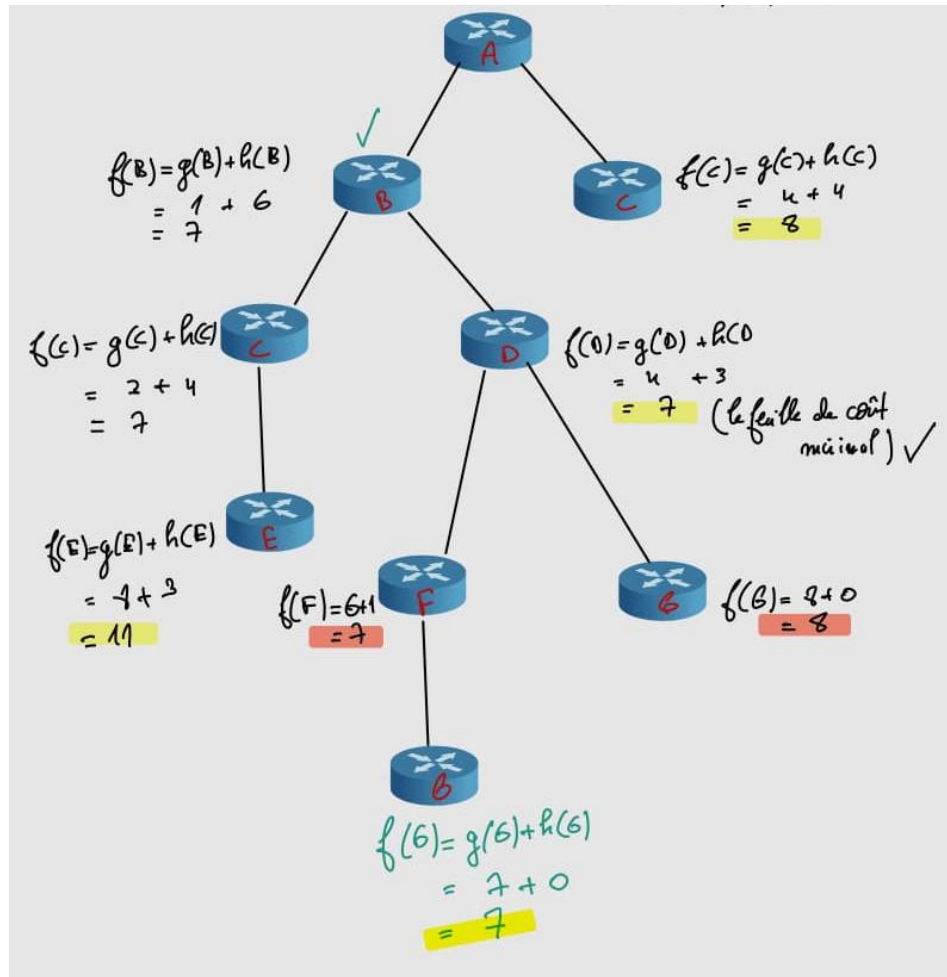
| Etape | Open List        | Closed List |
|-------|------------------|-------------|
| 0     | A(0)             | $\emptyset$ |
| 1     | B(7), C(8)       | A           |
| 2     | C(7), D(7), C(8) | A, B        |
|       |                  |             |
|       |                  |             |
|       |                  |             |



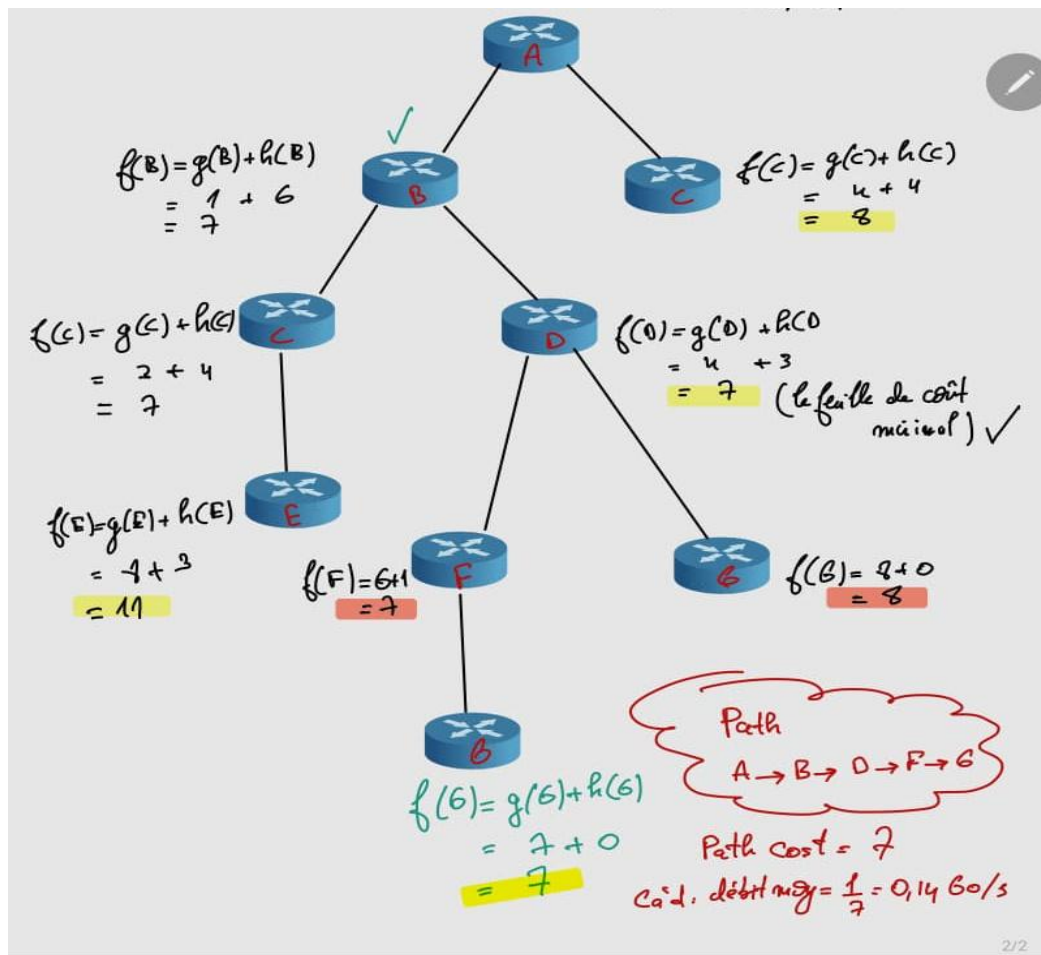
| Etape | Open List         | Closed List |
|-------|-------------------|-------------|
| 0     | A(0)              | $\emptyset$ |
| 1     | B(7), C(8)        | A           |
| 2     | C(7), D(7), C(8)  | A, B        |
| 3     | D(7), C(8), E(11) | A, B        |
|       |                   |             |
|       |                   |             |



| Etape | Open List               | Closed List |
|-------|-------------------------|-------------|
| 0     | A(0)                    | ∅           |
| 1     | B(7), C(8)              | A           |
| 2     | C(7), D(7), C(8)        | A, B        |
| 3     | D(7), C(8), E(11)       | A, B        |
| 4     | F(7), C(8), G(8), E(11) | A, B, D     |
|       |                         |             |



| Etape | Open List               | Closed List |
|-------|-------------------------|-------------|
| 0     | A(0)                    | $\emptyset$ |
| 1     | B(7), C(8)              | A           |
| 2     | C(7), C(8), D(7)        | A, B        |
| 3     | E(11), D(7), C(8)       | A, B        |
| 4     | F(7), C(8), G(8), E(11) | A, B, D     |
| 5     | G(7), C(8), G(8), E(11) | A, B, D, F  |



| Etape | Open List               | Closed List   |
|-------|-------------------------|---------------|
| 0     | A(0)                    | ∅             |
| 1     | B(7), C(8)              | A             |
| 2     | C(7), C(8), D(7)        | A, B          |
| 3     | E(11), D(7), C(8)       | A, B          |
| 4     | F(7), C(8), G(8), E(11) | A, B, D       |
| 5     | G(7), C(8), G(8), E(11) | A, B, D, F    |
| 6     | C(8), G(8), E(11)       | A, B, D, F, G |

# Partie Code

---

Les principales fonctions utiliser sont:

- **create\_grid()**  
Permet de créer la grille représentant le réseau ainsi que les obstacles.
- **create\_bandwidth()**  
Permet de générer les débits associés aux différents routeurs du réseau.
- **create\_distance()**  
Permet de générer les distances associées aux différents routeurs du réseau.
- **heuristic()**  
Calcule la distance de Manhattan utilisée comme heuristique dans l'algorithme A\*.
- **astar()**  
Implémente l'algorithme A\* afin de rechercher le chemin optimal entre la source et la destination.
- **simulate\_path()**  
Permet de simuler visuellement l'évolution du chemin trouvé par l'algorithme.

# Résultats et Simulation

---

Un test dynamique sera réalisé sous Google Colab afin de démontrer le fonctionnement de l'algorithme A\* dans le réseau simulé.



# Conclusion

---

Ce projet nous a permis de mieux comprendre le fonctionnement de l'algorithme A\* à travers son application dans un problème de routage réseau. Nous avons appris à modéliser un réseau sous forme de graphe et de grille, puis à implémenter l'algorithme en Python afin de rechercher le chemin optimal entre une source et une destination. L'intégration du coût, du débit et des obstacles nous a également permis de simuler un environnement réseau plus réaliste et d'observer le fonctionnement dynamique de l'algorithme sous Google Colab.

---

Merci De Votre Attention

---